AI Business Analyst Assistant

Technical Documentation

Akashe123

July 29, 2026

Contents

1 Introduction

The **AI Business Analyst Assistant** is an intelligent web application that bridges the gap between natural language and data analysis. It allows business users to upload their data (CSV/Excel files) and ask analytical questions in plain English, receiving instant SQL queries, visualizations, and actionable insights.

Business users traditionally spend hours waiting for reports or writing complex SQL queries. This system eliminates that bottleneck by leveraging Large Language Models (LLMs) to automatically translate natural language into database queries, execute them, and present results in an intuitive interface.

1.1 Problem Statement

Business users face several challenges in data analysis:

- **Technical Barriers:** Require SQL knowledge to query databases.
- **Time Constraints:** Waiting for IT teams to generate reports.
- **Data Overload:** Difficulty extracting meaningful insights from raw data.
- **Visualization Gaps:** Need separate tools for charting and analysis.

1.2 Solution Overview

Our application addresses these challenges by providing:

- A modern web interface for uploading data files (CSV/Excel).
- Natural language processing to understand business questions.
- Automatic SQL query generation using AI.
- Real-time data visualization with interactive charts.
- AI-generated business insights and recommendations.

2 System Architecture

The system follows a modern three-tier architecture:

2.1 Architecture Overview

Tier	Components
Frontend	Next.js 14, React 18, Recharts, TypeScript
Backend	FastAPI, LangChain, Groq AI, Pandas
Database	Neon (Serverless PostgreSQL), SQLAlchemy
Deployment	Vercel (Frontend + Backend)

2.2 Data Flow

The data flow follows this sequence:

1. **User Input:** User uploads a CSV/Excel file or asks a question.
2. **AI Processing:** Groq AI (LLaMA 3.3-70B) processes the natural language question.
3. **SQL Generation:** The AI generates a SQL query based on the database schema.
4. **Query Execution:** The query runs against the Neon PostgreSQL database.
5. **Analysis:** Results are analyzed for statistics and patterns.
6. **Insights:** AI generates business insights from the analyzed data.
7. **Visualization:** Charts are rendered using Recharts.
8. **Display:** Results are presented in the modern dark-theme UI.

3 Technology Stack

3.1 Frontend

- **Next.js 14:** React framework with server-side rendering.
- **React 18:** UI component library.
- **Recharts:** Composable charting library built on D3.js.
- **TypeScript:** Type-safe JavaScript.

3.2 Backend

- **FastAPI:** High-performance Python web framework.
- **LangChain:** Framework for LLM-powered applications.
- **Groq API:** Fast inference for LLaMA 3.3-70B model.
- **Pandas:** Data manipulation and analysis.

3.3 Database

- **Neon:** Serverless PostgreSQL with branching and autoscaling.
- **SQLAlchemy:** Python SQL toolkit and ORM.
- **psycopg2:** PostgreSQL adapter for Python.

4 Installation and Setup

4.1 Prerequisites

- Python 3.11 or higher
- Node.js 18 or higher
- A Groq API key (free at `https://console.groq.com/keys`)
- A Neon database (free at `https://console.neon.tech`)

4.2 Backend Setup

Listing 1: Backend Installation

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

4.3 Environment Configuration

Create a `.env` file in the `backend/` directory:

Listing 2: Environment Variables

```
GROQ_API_KEY=gsk_your_groq_api_key_here
DATABASE_URL=postgresql://user:pass@ep-xxx.aws.neon.tech/neondb?
   sslmode=require
```

4.4 Frontend Setup

Listing 3: Frontend Installation

```
cd frontend
npm install
```

4.5 Running the Application

Open two terminals:

Listing 4: Start Backend (Terminal 1)

```
cd backend
venv\Scripts\activate
uvicorn main:app --reload --port 8000
```

Listing 5: Start Frontend (Terminal 2)

```
cd frontend
npm run dev
```

Access the application at `http://localhost:3000`.

5 API Reference

5.1 Endpoints

Method	Endpoint	Description
POST	/api/ask	Ask a business question
POST	/api/upload	Upload CSV/Excel file
GET	/api/tables	List database tables
GET	/api/schema	Get database schema
GET	/api/health	Health check

5.2 POST /api/ask

Listing 6: Request

```
{
    "question": "Show me total sales by region"
}
```

Listing 7: Response

```
{
    "sql": "SELECT region, SUM(sales_amount) ...",
    "data": [...],
    "columns": ["region", "total_sales"],
    "insights": "1. Key finding: ...",
    "chart_recommendation": "bar"
}
```

6 Screenshots

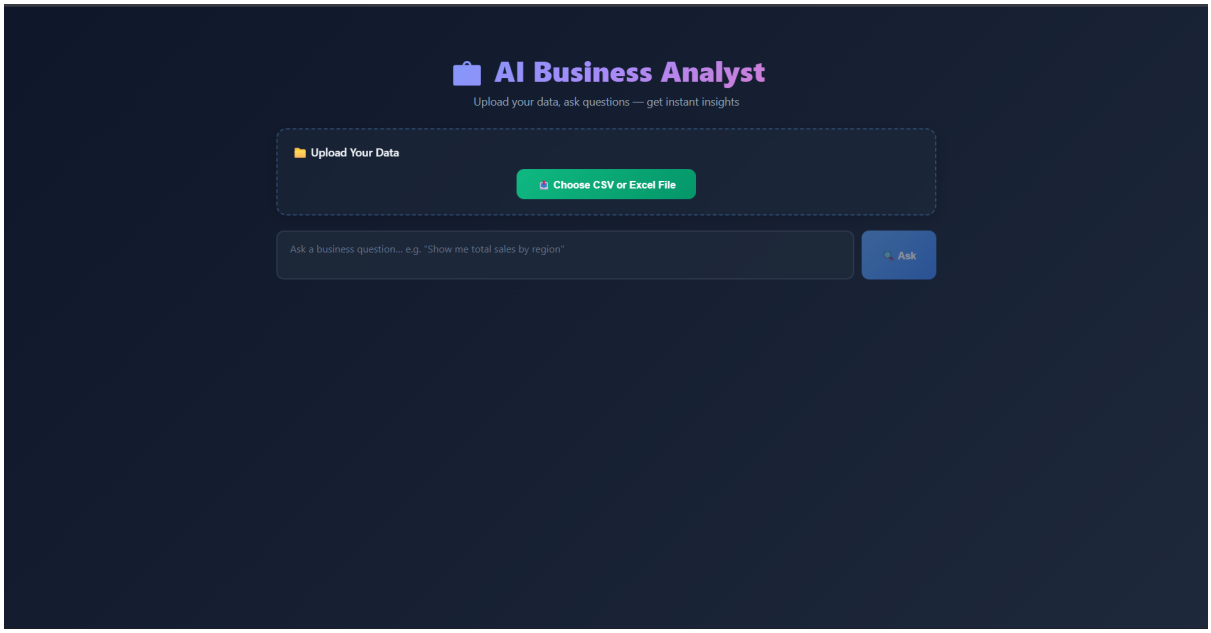

Figure 1: Main Dashboard

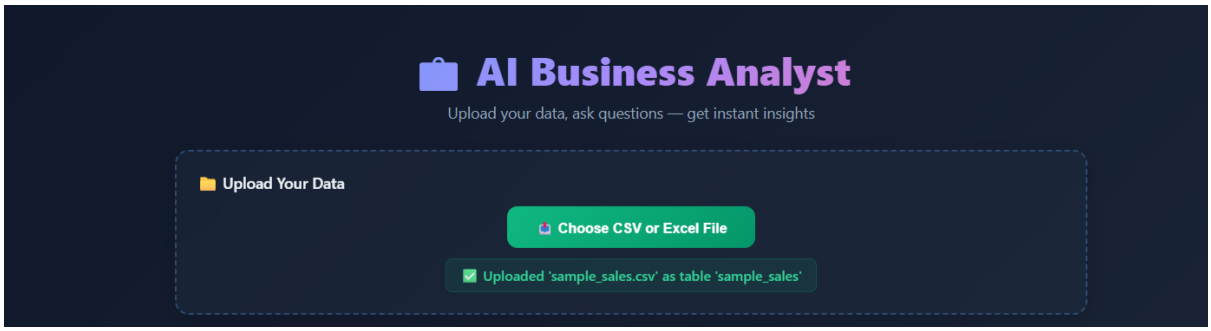

Figure 2: File Upload

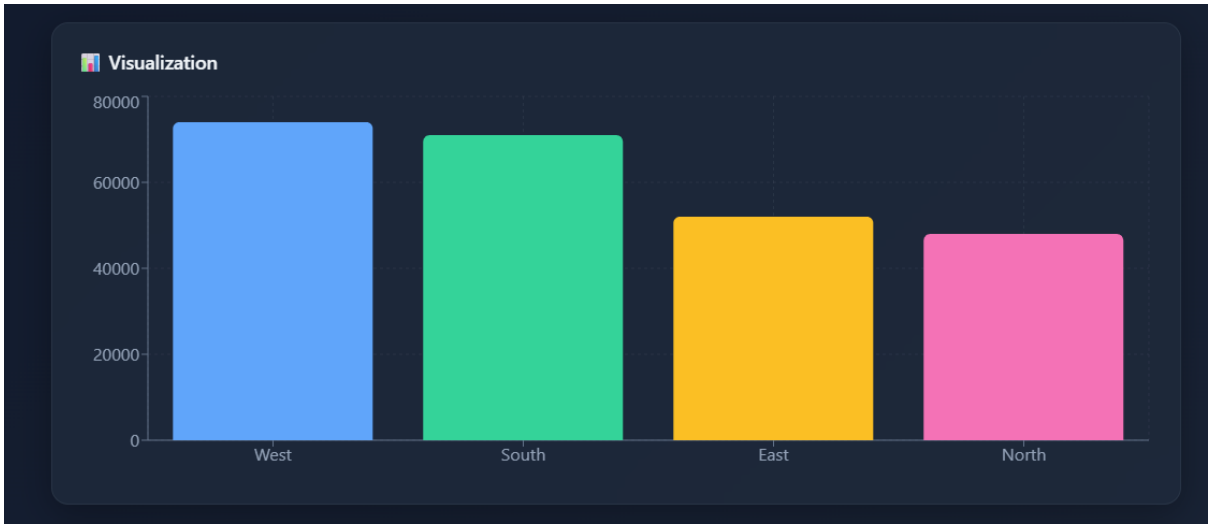

Figure 3: Interactive Charts

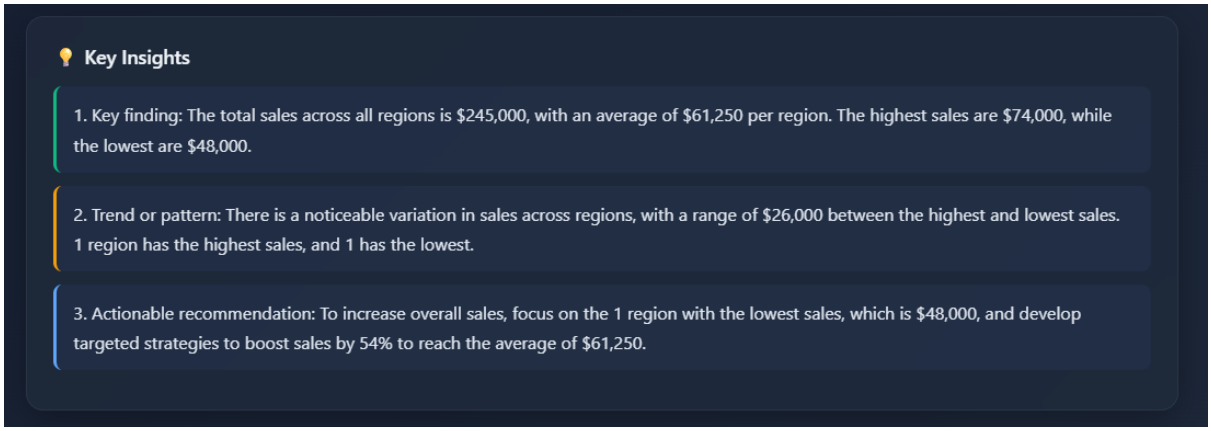

Figure 4: AI-Generated Insights

7 Deployment

7.1 Deploy to Vercel

Both frontend and backend can be deployed to Vercel:

Listing 8: Deploy Backend

```
cd backend
npm i -g vercel
vercel --prod
```

Set environment variables in Vercel dashboard: GROQ_API_KEY, DATABASE_URL.

Listing 9: Deploy Frontend

```
cd frontend
vercel --prod
```

Set environment variable: NEXT_PUBLIC_API_URL = your backend URL.

8 Conclusion

The AI Business Analyst Assistant successfully demonstrates how Large Language Models can democratize data analysis. By combining natural language processing, automated SQL generation, and interactive visualization, the system enables business users to derive insights from their data without technical expertise.